PROYECTO OPTATIVO

DOCUMENTO PLANIFICACIÓN Y RESULTADO FINAL

DAM1-ASIX1

El objetivo es que cada equipo entregue un documento profesional que deje constancia clara de qué va a desarrollar, cómo lo va a hacer y qué deberá tener terminado al final del curso.

No debéis limitaros a repetir la presentación inicial. Este documento debe concretar el proyecto con suficiente detalle para poder comparar, durante el curso, lo planificado con el avance real.

1. Resumen ejecutivo

- Nombre del proyecto y frase breve que explique qué vais a crear. CitaCan: Una app para pedir cita en la peluquería canina sin llamar, y una web para que el dueño gestione su negocio desde un solo sitio.

- Descripción general del proyecto en pocas líneas.

El cliente tendrá una app que básicamente buscara las peluquerías caninas mas cernas a el i le permitirá pedir cita viendo los huecos que le quedan a la peluqueria i además puede hacer una fixa a su perro (raza, peso, altura etc…) para así cuando reserve hora el peluquero pueda echarle una ojeada i así saber como es. I después tendríamos una web para el dueño que básicamente hay tendría un calendario con todas las citas y un resumen del mes de tema facturas i tal.

- Problema o necesidad que resuelve. Resuelve una desorganizacion de un pequeño negocio específicamente las peluquerías caninas haciendo una app para que los clientes se apunten ellos mismos i así el dueño no este pendiente todo el rato al teléfono i para los clientes básicamente podría buescar peluquerías cercanas a donde e

Al cliente lo que le resuelve es que puede ver peluquerías cercanas i puede reservar fácilmente una cita sin que haga falta en ponerse en contacto con el dueño. I el dueño no tendría que estar todo el rato pendiente del telefono

- Usuario o cliente principal. Gente que tenga un perro como mascota
- Resultado final que debería existir al terminar el curso. Pues a final de curso tendríamos que tener una app funcional con un registro funcional como de los clientes como de sus mascotas con una ficha del animal, después también una web para el dueño de la peluqueria con una agenda y una gestión de clientes, una base de datos. I si nos diera tiempo a todo haríamos lo siguiente: Recordatorio en el móvil, ingresos del mes, cancelar o editar citas, fotos del antes i después etc

2. Problema y justificación

- Explicar qué problema existe actualmente.

El problema que hemos encontrado es que en las peluquerías caninas solo se puede pedir hora si llamas o envias un mensaje y puede ser que cuando llames, estén cortando el pelo y tengan que para para coger el teléfono y apuntar

- Indicar quién tiene ese problema y en qué situaciones aparece.

El problema lo tienen la mayoría de peluquerías de perros, por ejemplo yo tengo un perro y para que le corten el pelo he de llamar, pedir hora y que se lo apunten en una agenda

- Explicar cómo se resuelve actualmente, si existe alguna solución.

No existen aplicaciones que hagan lo que queremos hacer nosotros, todas son de craer tu negocio y que la gente te diga cuando quiere ir y tu apuntarlo en la aplicacion

- Detallar las limitaciones de la situación actual.

Las limitaciones que podríamos tener es la limitación de conocimiento

- Justificar por qué merece la pena desarrollar vuestro proyecto.

Merece la pena porque es un proyecto que le podría servir a mucha gente y le facilitaría las cosas a muchas peluquerías ya que haciendo todo a papel, puedes perder las cosas y tenerlas desordenadas y con nuestro proyecto estará todo ordenado y fácil de ver

3. Usuarios y contexto de uso

- Definir el usuario principal y los usuarios secundarios.
- Indicar dónde y cuándo se utilizará el proyecto.
- Explicar desde qué dispositivos o entornos se utilizará.
- Describir las necesidades principales del usuario.
- Indicar qué nivel de conocimientos técnicos se espera del usuario.

CitaCan tiene dos usuarios que se encuentran en una misma cita: el cliente, que la pide, y el dueño de la peluquería, que la atiende

Usuario principal: el cliente. Es una persona que tiene un perro y busca una peluquería de confianza cerca de casa..Quiere reservar rápido y a la hora que le vaya bien, sin tener que llamar

Usuario secundario: el dueño de la peluquería. Es un negocio pequeño, de 1 a 3 personas, sin nadie en recepción. Mientras está bañando o cortando a un perro no puede coger el teléfono, y necesita tener su agenda ordenada

Dónde y cuándo se usará:

- El cliente usará la app desde cualquier sitio y a cualquier hora, por ejemplo en el sofá por la noche o en el trabajo, cuando se acuerde de que a su perro le toca peluquería

- El dueño usará la web en la peluquería, sobre todo por la mañana para mirar las citas del día y entre perro y perro para ver quién viene después

Dispositivos:

- Cliente: un móvil Android con conexión a internet

- Dueño: un ordenador o portátil con un navegador (Chrome, Firefox o Edge)

Necesidades principales:

Cliente	Dueño
Reservar en un minuto, sin llamar	No tener que estar pendiente del teléfono
Ver las horas libres de verdad	Ver su agenda de un vistazo
Tener la ficha de su perro guardada	Saber cómo es el perro antes de que llegue
Que le recuerden la cita	Cambiar sus servicios, precios y horario fácilmente

Nivel de conocimientos que esperamos: bajo. Tanto el cliente como el dueño solo tienen que saber usar un móvil o un ordenador a nivel básico, como cuando usan WhatsApp o miran una página web. Por eso la app y la web tienen que ser muy sencillas y con pocos pasos

4. Objetivos del proyecto

- Definir entre 3 y 6 objetivos concretos.
- Los objetivos deben poder comprobarse al final del proyecto.
- Evitar objetivos genéricos como «hacer una buena aplicación» o «mejorar el sistema».
- Indicar cómo se podrá saber si cada objetivo se ha conseguido.

1. Que la gente no tenga que llamar para pedir hora. El cliente tiene que poder reservar una cita desde la app eligiendo el servicio, el día y la hora, sin hablar con el dueño.

   Cómo lo comprobaremos: haremos una reserva desde el móvil de principio a fin sin llamar a nadie, y la cita tiene que quedar guardada.

2. Hacer una aplicación para el móvil. Una app Android donde el cliente se pueda registrar, iniciar sesión y crear la ficha de su perro (raza, tamaño, peso y notas).

   Cómo lo comprobaremos: instalaremos la app en un móvil Android, crearemos un usuario nuevo y le añadiremos un perro.

3. Crear una base de datos. Una base de datos donde se guarden los clientes, los perros, los servicios y las citas, y que use tanto la app como la web.

   Cómo lo comprobaremos: después de hacer una reserva, miraremos la base de datos y tiene que aparecer la cita con su cliente y su perro.

4. Hacer una web para el dueño con su agenda. El dueño tiene que ver en el ordenador las citas del día y de la semana, y las reservas nuevas tienen que aparecer sin que nadie le avise.

   Cómo lo comprobaremos: reservaremos desde el móvil y, al actualizar la web, la cita tiene que salir en la agenda.

5. Que el dueño pueda gestionar su negocio desde la web. Añadir, editar y borrar clientes, cambiar los servicios y precios, y marcar su horario y los días que cierra.

   Cómo lo comprobaremos: cambiaremos el precio de un servicio y cerraremos un día desde la web, y en la app tiene que salir el precio nuevo y ese día no se tiene que poder reservar.

5. Alcance del proyecto

- Describir todas las funcionalidades, módulos, servicios o componentes que sí se desarrollarán.
- Indicar qué partes son imprescindibles y cuáles son secundarias.
- Definir qué funcionalidades quedan expresamente fuera del proyecto.
- Distinguir entre funciones obligatorias y posibles ampliaciones si sobra tiempo.

CitaCan tendrá tres partes que trabajan juntas: la app del cliente, la web del dueño y un servidor con la base de datos que conecta las dos. Hemos separado lo que es imprescindible de lo que haremos solo si nos sobra tiempo, y también lo que no haremos

Parte	Imprescindible (MVP)	Si da tiempo (ampliaciones)
App del cliente (Android)	Registro e inicio de sesión, ficha del perro (raza, tamaño, peso y notas), elegir servicio, ver horas libres y reservar, ver mis citas	Cancelar o cambiar una cita, recordatorio en el móvil el día antes, buscar peluquerías cercanas
Web del dueño	Inicio de sesión, agenda del día y de la semana, añadir, editar y borrar clientes, cambiar servicios y precios, horario y días de cierre	Resumen de ingresos del mes, fotos del antes y el después de cada perro
Servidor y base de datos	Un servidor común para la app y la web, base de datos MySQL con usuarios, perros, servicios, citas y horario	Copias de seguridad automáticas

Lo que queda fuera del proyecto (no lo haremos):

- Pagos online en la app: el cliente pagará en la peluquería como siempre. Meter pagos es complicado y necesita más seguridad
- Publicar la app en Google Play: la instalaremos directamente en el móvil para las pruebas y la demo
- Versión para iPhone: solo haremos Android, que es lo que podemos programar y probar con lo que tenemos
- Varios negocios en la misma plataforma: CitaCan funcionará para una sola peluquería. Que el cliente pueda elegir entre varias peluquerías lo dejamos como idea para el futuro

Solo empezaremos con las ampliaciones cuando todo lo imprescindible funcione de principio a fin

6. MVP o versión mínima viable

- Definir cuál es la versión mínima que debe funcionar obligatoriamente.
- Indicar qué funciones deben estar operativas para considerar que el proyecto existe realmente.
- Explicar qué se deberá poder demostrar en directo con el MVP.
- Diferenciar claramente el MVP de las ampliaciones opcionales.

Lo que tiene que funcionar obligatoriamente:

- App del cliente (Android): registrarse, iniciar sesión, crear la ficha del perro, elegir un servicio, ver solo las horas libres y reservar.
- Web del dueño: iniciar sesión, ver la agenda del día y de la semana, gestionar los clientes (añadir, editar y borrar), cambiar los servicios y precios, y poner el horario y los días de cierre.
- Servidor y base de datos: un servidor común que conecta la app y la web, y una base de datos MySQL donde se guarda todo.

Lo que enseñaremos en directo con el MVP:

1. Desde el móvil, creamos un cliente nuevo y le añadimos un perro.
2. Elegimos un servicio y nos salen solo las horas que están libres.
3. Reservamos una hora.
4. En el ordenador, entramos en la web del dueño y la cita aparece en la agenda.
5. Si otro cliente intenta reservar a la misma hora, ya no le sale disponible.

Lo que NO entra en el MVP: cancelar o cambiar una cita desde la app, el recordatorio en el móvil el día antes, el resumen de ingresos del mes y las fotos del antes y el después. Solo empezaremos con estas cosas cuando el MVP funcione entero.

7. Requisitos funcionales

- Enumerar las acciones que el sistema deberá permitir realizar.
- Escribir los requisitos de forma concreta y comprobable.
- Identificar qué usuario realiza cada acción cuando sea necesario.
- Ordenar o priorizar los requisitos si el proyecto tiene muchas funcionalidades.

Estas son las acciones que tendrá que permitir CitaCan, quién las hace y su prioridad. Alta quiere decir que es parte del MVP y Baja que es una ampliación.

Nº	Requisito	Usuario	Prioridad
RF1	Registrarse con nombre, correo, teléfono y contraseña	Cliente	Alta
RF2	Iniciar y cerrar sesión	Cliente y dueño	Alta
RF3	Crear, editar y borrar la ficha de un perro (nombre, raza, tamaño, peso y notas)	Cliente	Alta
RF4	Ver la lista de servicios con su precio y duración	Cliente	Alta
RF5	Ver solo las horas libres de un día, según el horario y las citas que ya hay	Cliente	Alta
RF6	Reservar una cita eligiendo perro, servicio, día y hora	Cliente	Alta
RF7	Ver sus citas pendientes	Cliente	Alta
RF8	Ver la agenda del día y de la semana con todas las citas	Dueño	Alta
RF9	Ver la ficha del perro de cada cita	Dueño	Alta
RF10	Añadir, editar y borrar clientes	Dueño	Alta
RF11	Añadir, editar y borrar servicios y cambiar sus precios	Dueño	Alta
RF12	Poner el horario de apertura y los días de cierre	Dueño	Alta
RF13	No dejar reservar dos citas a la misma hora	Sistema	Alta
RF14	Cancelar o cambiar una cita	Cliente	Baja
RF15	Recibir un recordatorio en el móvil el día antes de la cita	Cliente	Baja
RF16	Ver un resumen de las citas y los ingresos del mes	Dueño	Baja
RF17	Subir fotos del antes y el después del perro	Dueño	Baja

8. Requisitos no funcionales

- Rendimiento esperado.
- Seguridad y control de acceso.
- Usabilidad y accesibilidad.
- Compatibilidad con dispositivos o sistemas.
- Disponibilidad y fiabilidad.
- Privacidad y tratamiento de datos.
- Mantenibilidad y documentación.

Lo que esperamos es tener una aplicación que funcione bien, que sea segura, que sea fácil de usar y que la pueda entender todo el mundo, que no dé problemas, y para la página del dueño, que sea fácil de utilizar, fácil de entender y que sea segura.

- Rendimiento: las pantallas tienen que cargar rápido, en unos 2 o 3 segundos como mucho. Como será una sola peluquería con pocos clientes a la vez, no necesitamos nada especial.
- Seguridad y control de acceso: cada usuario entra con su correo y su contraseña. Habrá dos tipos de usuario: el cliente, que solo ve sus perros y sus citas, y el dueño, que ve toda la agenda. Las contraseñas se guardarán cifradas.
- Usabilidad y accesibilidad: pocos botones, textos grandes y claros, y que reservar una cita no lleve más de cuatro pasos. Pensamos en gente que no se maneja mucho con el móvil
- Compatibilidad: la app será solo para Android (no haremos versión de iPhone). La web del dueño tiene que funcionar en Chrome, Firefox y Edge desde un ordenador
- Disponibilidad y fiabilidad: si dos personas intentan coger la misma hora, solo se la tiene que quedar uno. Si algo falla, la app tiene que mostrar un mensaje que se entienda y no cerrarse de golpe
- Privacidad y tratamiento de datos: solo pediremos los datos necesarios (nombre, correo, teléfono y los datos del perro), seguiremos lo que dice el RGPD y el usuario tendrá que aceptar que guardemos sus datos al registrarse
- Mantenibilidad y documentación: el código estará en GitHub, con nombres claros y comentarios en las partes más difíciles. Haremos un manual de instalación y otro de usuario

9. Casos de uso principales

- Describir al menos tres situaciones reales de uso.
- Indicar quién inicia cada acción.
- Explicar los pasos principales que realizará el usuario.
- Indicar cuál debe ser el resultado esperado.

Estas son cuatro situaciones reales en las que se usaría CitaCan.

Caso 1: Laura pide cita para su perro

- Quién lo inicia: el cliente (Laura).
- Situación: es domingo por la noche y Laura se da cuenta de que a Toby, su perro mediano, le hace falta un baño y un corte.
- Pasos:
    1. Abre la app e inicia sesión.
    2. Elige a Toby de su lista de perros.
    3. Elige el servicio "Baño + corte" y la app le dice que durará 1 hora y 30 minutos.
    4. Elige el martes y le salen solo las horas libres. Escoge las 11:30.
    5. Confirma la reserva.
- Resultado esperado: le sale el mensaje "¡Cita reservada!", la cita aparece en "Mis citas" y esa hora ya no está libre para nadie más.

Caso 2: El dueño mira su agenda del día

- Quién lo inicia: el dueño de la peluquería.
- Situación: es martes por la mañana y el dueño quiere saber qué perros vienen hoy.
- Pasos:
    1. Abre la web en el ordenador de la peluquería e inicia sesión.
    2. Entra en la agenda del día.
    3. Ve que a las 11:30 viene Toby y hace clic en la cita para ver su ficha.
- Resultado esperado: ve todas las citas ordenadas por hora y sabe que Toby es mediano y que no le gusta el secador, así se puede preparar antes de que llegue.

Caso 3: El dueño cierra por vacaciones y cambia un precio

- Quién lo inicia: el dueño de la peluquería.
- Situación: la peluquería cerrará un viernes por un festivo y, además, el dueño quiere subir el precio del corte.
- Pasos:
    1. En la web, entra en "Horario" y marca ese viernes como día de cierre.
    2. Entra en "Servicios y precios" y cambia el precio del corte.
    3. Guarda los cambios.
- Resultado esperado: en la app, ese viernes ya no se puede reservar y el corte sale con el precio nuevo.

Caso 4: Un cliente nuevo se registra

- Quién lo inicia: un cliente nuevo.
- Situación: Marc acaba de adoptar a Luna y quiere llevarla por primera vez a la peluquería.
- Pasos:
    1. Se descarga la app y pulsa "Registrarse".
    2. Pone su nombre, correo, teléfono y contraseña, y acepta la política de privacidad.
    3. Crea la ficha de Luna con su raza, tamaño, peso y una nota.
- Resultado esperado: Marc ya tiene su cuenta y la ficha de Luna guardada, y puede pedir cita como en el caso

10. Diseño y experiencia de usuario

- Definir las pantallas, interfaces, paneles, esquemas o elementos visuales principales.
- Explicar el recorrido habitual del usuario.
- Incluir bocetos, wireframes, Figma, planos o prototipos cuando corresponda.
- Explicar las decisiones principales de diseño y usabilidad.
- Indicar si existirá versión móvil, escritorio o ambas.

Habrá dos partes: una app para el móvil (Android) para los clientes y una web para el ordenador para el dueño. Los bocetos los haremos en Figma, que es gratis y fácil de usar, y ya tenemos un primer boceto en la presentación.

Pantallas de la app del cliente:

- Inicio de sesión y registro
- Mis perros: la lista de sus perros y la ficha de cada uno (nombre, raza, tamaño, peso y notas)
- Elegir servicio: baño, corte o baño + corte, con su precio y lo que dura
- Elegir día y hora: un calendario donde solo salen las horas libres
- Confirmación: un resumen de la cita y el botón de reservar
- Mis citas: las citas que tiene pendientes

Pantallas de la web del dueño:

- Inicio de sesión
- Agenda: las citas del día y de la semana, con el nombre del perro, el servicio y la hora
- Clientes: la lista de clientes con sus perros, para añadir, editar o borrar
- Servicios y precios: para cambiar lo que ofrece y cuánto cuesta
- Horario: las horas que abre y los días que cierra

Recorrido habitual del cliente: abre la app → inicia sesión → elige el perro → elige el servicio → elige un hueco libre → confirma la reserva → le sale el mensaje de "¡Cita reservada!"

Recorrido habitual del dueño: por la mañana abre la web en el ordenador de la peluquería → mira la agenda del día → si hace falta, mira la ficha del perro antes de que llegue

Decisiones de diseño:

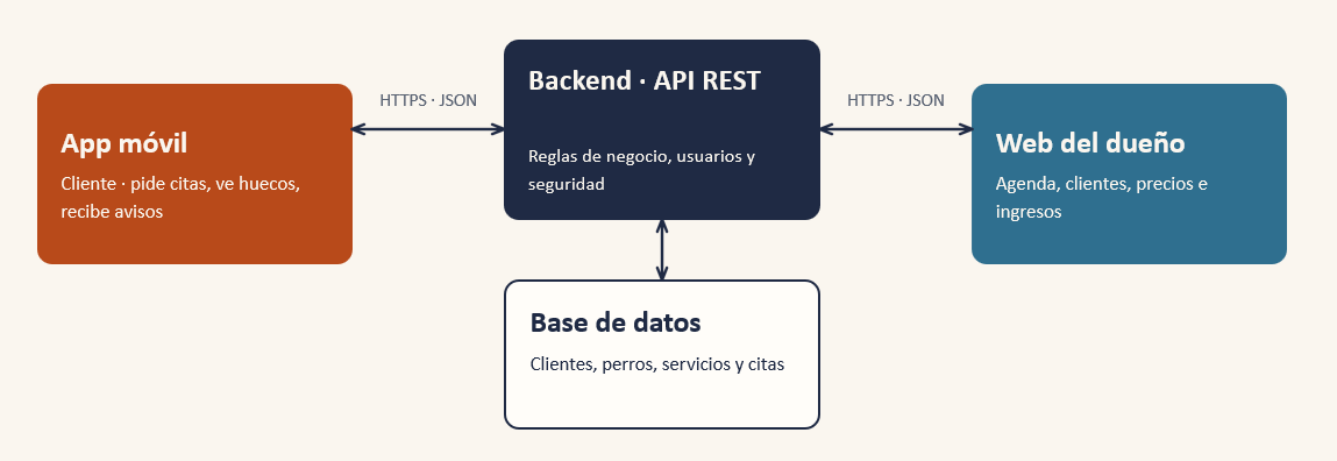
- Pocos pasos y un botón principal por pantalla, para que nadie se pierda
- Solo se enseñan las horas libres, así el cliente no puede elegir una que esté ocupada
- La app dice cuánto durará la cita según el servicio y el tamaño del perro

- Los mismos colores y el mismo logo en la app y en la web, para que se vea que es lo mismo
- Letra grande y colores con buen contraste para que se lea bien

11. Arquitectura técnica

- Explicar cómo estará construido el proyecto de principio a fin.
- Detallar frontend, backend, base de datos, servidor o hosting si corresponde.
- Explicar redes, comunicaciones, hardware, APIs e integraciones.
- Indicar cómo se realizará la monitorización y el registro de errores.
- Justificar las tecnologías elegidas y no limitarse a enumerarlas.
- Incluir un diagrama de arquitectura.

CitaCan tiene tres partes: la app del cliente, la web del dueño y un servidor que guarda todo en una base de datos. La app y la web no hablan entre ellas: las dos le piden las cosas al mismo servidor, y por eso cuando alguien reserva desde el móvil el dueño lo ve en la web al momento.



Las flechas de doble sentido indican que la información va y vuelve; la caja con borde de rayas es una ampliación.

Cómo funciona por dentro, de principio a fin:

1. El cliente pulsa "Reservar" en la app.
2. La app envía la petición al servidor por internet (HTTPS), con los datos en formato JSON.
3. El servidor comprueba quién es el usuario (con su token), mira si la hora sigue libre y guarda la cita en la base de datos.
4. El servidor contesta a la app que todo ha ido bien.
5. Cuando el dueño abre o actualiza la agenda en la web, la web pide las citas al servidor y le sale la nueva.

Tecnologías elegidas y por qué:

Parte	Tecnología	Por qué la elegimos
App del cliente (frontend móvil)	Android con Java, en Android Studio	Java es el lenguaje que damos en Programación, y Android Studio es gratis y tiene muchos tutoriales
Web del dueño (frontend web)	HTML, CSS y JavaScript	Es lo que vemos en Lenguajes de Marcas. No usamos React para no tener que aprender otra cosa más
Servidor (backend)	API REST con Java y Spring Boot	Así usamos Java también en el servidor y no tenemos que aprender otro lenguaje. Spring Boot ya trae casi todo hecho para crear una API
Base de datos	MySQL	Es la que damos en Bases de Datos, es gratis y la conocemos
Avisos al móvil (ampliación)	Firebase Cloud Messaging	Es gratis y es lo que usa Android para enviar notificaciones
Código	GitHub	Para tener el código guardado y trabajar los dos a la vez

Dónde estará el servidor: mientras lo programamos, cada uno lo tendrá en su portátil. Para las pruebas y la demo final lo pondremos en un servidor del instituto o, si no se puede, en un portátil que haga de servidor (lo explicamos en el apartado 13).

Registro de errores: el servidor guardará en un archivo (log) los errores que pasen, con la fecha y la hora, para que podamos ver qué ha fallado. En la app, si algo falla, se mostrará un mensaje claro al usuario en vez de cerrarse.

12. Datos y base de datos

- Explicar qué información almacenará el proyecto.
- Definir las principales entidades, tablas o colecciones.
- Explicar las relaciones entre los datos.
- Indicar qué información será obligatoria y cuál opcional.
- Definir quién podrá consultar, crear, modificar o eliminar datos.
- Explicar cómo se realizarán las copias de seguridad y la recuperación.
- Incluir un modelo de datos o diagrama entidad-relación si procede.

Usaremos MySQL, que es la base de datos que estamos aprendiendo en clase. Guardaremos los datos de los usuarios, de sus perros, de los servicios de la peluquería, de las citas y del horario.

Tablas principales:

Tabla	Qué guarda	Obligatorio	Opcional
Usuario	Las personas que usan CitaCan (clientes y dueño)	Nombre, correo, contraseña, teléfono, rol (cliente o dueño)	—

Perro	La ficha de cada perro	Nombre, raza, tamaño (pequeño, mediano o grande), dueño	Peso, edad, notas (alergias, si muerde, etc.)
Servicio	Lo que ofrece la peluquería	Nombre (baño, corte…), precio, duración en minutos	Descripción
Cita	Cada reserva	Perro, servicio, fecha, hora de inicio, hora de fin, estado	Notas del cliente
Horario	Cuándo abre la peluquería	Día de la semana, hora de apertura, hora de cierre	—
Día de cierre	Festivos o vacaciones	Fecha	Motivo

Relaciones entre los datos:

- Un usuario (cliente) puede tener varios perros, pero cada perro es de un solo usuario.
- Un perro puede tener varias citas, y cada cita es de un solo perro.
- Cada cita es de un servicio, y un servicio puede estar en muchas citas.
- El horario y los días de cierre sirven para saber qué horas están libres.

Quién puede hacer qué con los datos:

- Cliente: crea, ve, edita y borra sus propios datos y los de sus perros, y crea sus citas. No puede ver los datos de los otros clientes.
- Dueño: ve todas las citas y todos los clientes, puede añadir, editar y borrar clientes, y es el único que puede cambiar los servicios, los precios y el horario.

Copias de seguridad: una vez a la semana haremos una copia de la base de datos con la herramienta de exportar de MySQL (mysqldump o MySQL Workbench) y la guardaremos en Google Drive y en un USB. Si se pierde algo, importaremos la última copia. También tendremos en GitHub el script que crea las tablas, así podemos volver a montar la base de datos desde cero.

13. Infraestructura, sistemas y redes

- Detallar equipos físicos o virtuales necesarios.
- Indicar sistemas operativos.
- Describir servicios, puertos y comunicaciones.
- Explicar direccionamiento, VLAN, segmentación o topología si corresponde.
- Describir DNS, dominio, certificados y acceso remoto si se utilizan.
- Explicar virtualización, contenedores o servidores cuando corresponda.
- Detallar monitorización, copias y restauración.

La infraestructura de CitaCan es sencilla: nuestros portátiles para programar, un móvil Android para probar y un ordenador que haga de servidor. No necesitamos comprar nada.

Equipos necesarios:

Equipo	Para qué	Sistema operativo
2 portátiles (los nuestros)	Programar la app, la web y el servidor	Windows
1 móvil Android	Probar la app y hacer la demo	Android
Servidor (del instituto o un portátil nuestro)	Tener en marcha el servidor y la base de datos para las pruebas y la demo	Xampp
Emulador de Android Studio	Probar la app sin móvil o como plan B	—

Servicios y puertos:

- Servidor de CitaCan (Spring Boot): puerto 8080. Es por donde la app y la web le hacen las peticiones.
- MySQL: puerto 3306. Solo lo usa el servidor de CitaCan desde el mismo ordenador, no estará abierto a internet.
- HTTPS: puerto 443, si conseguimos ponerlo en un servidor con dominio.

Red y comunicaciones: mientras programamos y en la demo, el móvil y el servidor estarán conectados a la misma red wifi, y la app se conectará al servidor con su dirección IP. Como es un proyecto pequeño no necesitamos VLAN ni dividir la red. Si nos dejan un servidor del instituto, preguntaremos al profesorado cómo acceder a él y qué puertos podemos usar.

Dominio y certificados: para el MVP no hace falta dominio. Si nos da tiempo y ponemos el servidor en internet, buscaremos un dominio gratuito y un certificado gratis de Let's Encrypt para usar HTTPS.

Contenedores: si nos da tiempo, aprenderemos a usar Docker para meter el servidor y la base de datos en contenedores, así se podría instalar todo en cualquier ordenador con un solo comando. Si no, lo instalaremos a mano siguiendo nuestro manual de instalación.

Monitorización, copias y restauración: revisaremos el archivo de errores (log) del servidor cuando algo falle. Las copias de la base de datos se harán cada semana y se guardarán en Google Drive y en un USB (ver apartado 12). Para restaurar, instalaremos MySQL e importaremos la última copia, y el código lo volveremos a bajar de GitHub.

14. Seguridad y privacidad

- Explicar cómo se controlará el acceso al sistema.
- Definir roles y permisos si existen.
- Explicar cómo se protegerán contraseñas, claves y secretos.
- Describir medidas frente a pérdida de datos o caída de servicios.
- Indicar qué pruebas de seguridad se realizarán.

- Explicar si se tratarán datos personales y cuáles.
- Detallar quién tendrá acceso a los datos y dónde se almacenarán.
- Explicar cómo se evitará mostrar información sensible durante pruebas o demostraciones.

Nadie podrá usar CitaCan sin iniciar sesión con su correo y su contraseña, y cada usuario solo podrá ver y tocar lo que le toca según su rol.

Control de acceso y roles:

- Cliente: entra desde la app. Solo ve sus datos, sus perros y sus citas.
- Dueño: entra desde la web. Ve la agenda completa y los clientes, y puede cambiar servicios, precios y horario.
- Cuando alguien inicia sesión, el servidor le da un token (una especie de pase) que se envía en cada petición. El servidor mira ese token para saber quién es y qué rol tiene antes de dejarle hacer algo.

Contraseñas y claves:

- Las contraseñas no se guardan tal cual, sino cifradas (con BCrypt), así aunque alguien viera la base de datos no las podría leer.
- La contraseña de la base de datos y otras claves estarán en un archivo de configuración que no se sube a GitHub.
- La app, la web y el servidor se comunicarán por HTTPS para que los datos viajen protegidos.

Pérdida de datos o caída del servicio: haremos copias de seguridad de la base de datos cada semana (ver apartado 12). Si el servidor se cae, tenemos todo en GitHub y lo podemos volver a montar en un portátil.

Pruebas de seguridad que haremos:

- Intentar entrar con una contraseña incorrecta y comprobar que no deja.
- Entrar como cliente e intentar ver las citas de otro cliente o entrar en la web del dueño, y comprobar que no deja.
- Mirar en la base de datos que las contraseñas salen cifradas.

Datos personales que tratamos: del cliente, nombre, correo y teléfono; del perro, nombre, raza, tamaño, peso y notas. No guardaremos datos de pago ni de tarjetas. Los datos estarán en la base de datos del servidor y solo tendrán acceso el dueño de la peluquería y nosotros dos como desarrolladores. Al registrarse, el usuario tendrá que aceptar que guardemos sus datos, como pide el RGPD.

En las pruebas y en la demo: usaremos datos inventados (clientes y perros de prueba como "Laura" y "Toby"), nunca datos reales de clientes de la peluquería, y no enseñaremos contraseñas ni el archivo de configuración en pantalla.

15. Recursos, costes y dependencias

- Indicar qué hardware ya está disponible y qué material habrá que conseguir.
- Detallar software, plataformas, servicios o cuentas necesarias.
- Indicar si existirán costes de hosting, dominio, licencias o material.
- Estimar el coste total del proyecto.
- Identificar dependencias externas: empresas, APIs, permisos, servicios, dispositivos o cuentas.
- Definir una alternativa para las dependencias críticas.

El coste previsto del proyecto es de 0 €, porque todo el material ya lo tenemos y todos los programas que usaremos son gratis.

Hardware:

- Ya lo tenemos: nuestros dos portátiles y un móvil Android (el de uno de nosotros).

- Hay que conseguir: un ordenador que haga de servidor para la demo. Lo pediremos al instituto y, si no puede ser, usaremos uno de nuestros portátiles.

Software, servicios y cuentas:

Herramienta	Para qué	Coste
Android Studio	Programar la app	Gratis
Visual Studio Code	Programar la web	Gratis
IntelliJ IDEA Community	Programar el servidor con Spring Boot	Gratis
MySQL y MySQL Workbench	La base de datos	Gratis
GitHub	Guardar el código y trabajar los dos a la vez	Gratis
Figma	Hacer los bocetos de las pantallas	Gratis (plan para estudiantes)
Trello	Organizar las tareas	Gratis
Firebase (ampliación)	Enviar los avisos al móvil	Gratis

Costes que podría haber: si al final quisiéramos poner el servidor en internet con un servidor alquilado (VPS) y un dominio, costaría unos 5 € al mes más el dominio. No está previsto, porque para la demo no hace falta.

Dependencias externas y alternativa:

Dependencia	Qué pasa si falla	Alternativa

Una peluquería canina que nos ayude a probarlo	No podríamos validar la idea con un negocio real	Hacer las pruebas con familiares, compañeros y profesorado, y preguntar a varias peluquerías
El servidor del instituto	No tendríamos dónde tener el servidor para la demo	Usar uno de nuestros portátiles como servidor
El móvil Android	No podríamos probar ni enseñar la app	Usar el emulador de Android Studio
Firebase (avisos)	No funcionarían los recordatorios	Es una ampliación, el MVP funciona sin ella

16. Organización del equipo

- Indicar los integrantes del equipo.
- Asignar una responsabilidad principal a cada miembro.
- Definir responsabilidades secundarias.
- Indicar quién coordina el proyecto.
- Indicar quién mantiene la documentación.
- Explicar cómo se redistribuirá el trabajo si alguien no puede completar su parte.

Somos dos: Albert Molinas y Pol de la Viuda. Nos hemos repartido el trabajo para que cada uno se encargue de una parte, pero nos ayudaremos en todo.

Integrante	Responsabilidad principal	Responsabilidades secundarias
Albert Molinas	Coordinación del proyecto, diseño (Figma) y app móvil del cliente	Plan de sprints, repositorio de GitHub, pruebas de la app
Pol de la Viuda	Servidor (backend), base de datos y web del dueño	Seguridad, poner el servidor en marcha, pruebas de la web

- Quién coordina: Albert. Se encarga de que se cumplan los sprints y de repartir las tareas.
- Quién mantiene la documentación: entre los dos. Cada uno documenta lo que hace, y al final de cada sprint repasamos juntos el documento para que esté al día.
- Si alguien no puede acabar su parte: lo diremos en cuanto lo veamos, sin esperar al final del sprint. El otro le ayudará o se quedará esa tarea, y si aun así no llegamos, quitaremos antes una ampliación que una parte del MVP. Como los dos tendremos el código en GitHub, cualquiera puede continuar el trabajo del otro.

Los roles pueden cambiar durante el curso si vemos que a uno se le da mejor una parte.

17. Metodología de trabajo

- Indicar la herramienta utilizada para gestionar tareas.
- Definir cómo se utilizará el repositorio o control de versiones si corresponde.
- Indicar la frecuencia de reuniones del equipo.
- Explicar cómo se asignarán y revisarán las tareas.
- Explicar cómo se documentarán las decisiones y los cambios.
- Definir cómo se integrarán las diferentes partes del proyecto.

Trabajaremos por sprints de 2 semanas, organizando las tareas en Trello y guardando el código en GitHub.

Gestión de tareas (GitHub): tendremos un tablero con cuatro columnas: *Por hacer*, *Haciendo*, *En revisión* y *Hecho*. Cada tarea será una tarjeta con el nombre de quien la hace y el sprint al que pertenece.

Repositorio (GitHub):

- Tendremos un solo repositorio con tres carpetas: app, web y servidor.
- La rama main siempre tendrá una versión que funcione.
- Para cada tarea nueva crearemos una rama aparte (por ejemplo, reservar-cita) y, cuando esté acabada, la uniremos a main.
- Haremos commits a menudo, con mensajes que expliquen lo que hemos cambiado.

Reuniones:

- Dos veces por semana, en clase, haremos una reunión corta de 10 minutos para ver qué ha hecho cada uno, qué hará ahora y si tiene algún problema.
- Al final de cada sprint, una reunión más larga para revisar lo que hemos hecho y planificar el siguiente sprint.
- Entre reuniones hablaremos por WhatsApp.

Cómo se asignan y revisan las tareas: al empezar cada sprint, Albert reparte las tareas en Trello según la parte de cada uno. Cuando alguien acaba una tarea, la pasa a *En revisión* y el otro la prueba antes de pasarla a *Hecho*.

Cómo documentamos las decisiones y los cambios: las decisiones importantes las apuntaremos en este documento, en el apartado 25 (Control de cambios), con la fecha y el motivo. Los cambios del código quedan guardados en el historial de GitHub.

Cómo juntamos las partes: antes de programar, apuntaremos en un documento qué peticiones tendrá el servidor (por ejemplo, "pedir horas libres" o "crear cita") y qué datos envía y recibe cada una. Así Albert puede hacer la app y Pol el servidor a la vez, sabiendo cómo se conectarán. Al final de cada sprint probaremos todo junto para ver que encaja.

18. Plan de trabajo y sprints

- Dividir el proyecto en fases o sprints(Sabemos las semanas que tenemos de curso, y cada Sprint es cada 2 semanas)
- Definir el objetivo de cada sprint.
- Indicar qué entregable demostrable deberá existir al final de cada fase.
- Indicar fechas previstas de finalización.
- Como mínimo contemplar: análisis, diseño, primera versión funcional, desarrollo principal, integración, pruebas y cierre.

Dividimos el proyecto en 13 sprints de 2 semanas, desde mediados de octubre hasta finales de mayo, sin contar las vacaciones de Navidad y Semana Santa. Las fechas son aproximadas y las ajustaremos al calendario del curso.

Sprint	Fase	Objetivo	Entregable al final	Fin previsto
1	Análisis	Definir los requisitos y los casos de uso	Lista de requisitos y casos de uso	30/10/2026
2	Diseño	Hacer las pantallas en Figma y diseñar la base de datos	Prototipo en Figma y diagrama de la base de datos	13/11/2026
3	Preparación	Instalar las herramientas, crear el repositorio de GitHub, crear la base de datos y hacer pruebas pequeñas	Base de datos creada y un "hola mundo" de la app y del servidor	27/11/2026
4	Primera versión	Servidor con registro e inicio de sesión	Poder registrarse e iniciar sesión desde la app	18/12/2026
5	Primera versión	Ficha del perro y lista de servicios en la app	Crear un perro desde la app y ver los servicios	22/01/2027
6	Primera versión (MVP)	Ver horas libres y reservar	Reservar una cita desde la app y que se guarde	05/02/2027
7	Desarrollo principal	Web del dueño: inicio de sesión y agenda	Ver en la web las citas reservadas desde la app	19/02/2027
8	Desarrollo principal	Web del dueño: clientes, servicios, precios y horario	Gestionar el negocio desde la web	05/03/2027
9	Integración	Conectar bien todo y arreglar errores	MVP completo funcionando de principio a fin	19/03/2027
10	Ampliaciones	Cancelar o cambiar citas y resumen de ingresos del mes (si da tiempo)	Ampliaciones que hayamos podido hacer	16/04/2027
11	Pruebas	Pruebas funcionales, de seguridad y con usuarios reales	Documento con las pruebas y sus resultados	30/04/2027
12	Pruebas y mejoras	Arreglar lo que salga en las pruebas y el recordatorio si da tiempo	Versión final estable	14/05/2027
13	Cierre	Manuales, documentación final, presentación y ensayo de la demo	Todos los entregables finales	28/05/2027

Al final de cada sprint nos sentaremos los dos a ver qué hemos hecho, qué se ha quedado sin hacer y si hay que cambiar algo del plan.

19. Riesgos y plan B

- Identificar al menos cinco riesgos reales.
- Valorar su probabilidad e impacto.
- Explicar cómo intentaréis prevenir cada riesgo.

- Definir qué haréis si finalmente ocurre.
- Incluir riesgos de tiempo, tecnología, integraciones, hardware, permisos, conocimientos, costes o dependencia de terceros cuando correspondan.

El riesgo más grande es **que nos falten conocimientos y tiempo**, porque es la primera aplicación que hacemos. Estos son los riesgos que vemos, lo probables que son, cuánto nos afectarían y qué haremos.

Riesgo	Probabilidad	Impacto	Cómo lo prevenimos	Qué haremos si pasa
Conocimientos: no saber programar alguna parte (Android, Spring Boot)	Alta	Alto	Dedicar el sprint 3 a aprender y hacer pruebas pequeñas, y seguir tutoriales desde el principio	Pedir ayuda al profesorado, simplificar esa parte o cambiar a una tecnología más fácil
Tiempo: no llegar a acabarlo todo	Alta	Alto	Hacer primero el MVP y dejar las ampliaciones para el final; revisar el avance en cada sprint	Quitar ampliaciones; el MVP no se toca
Integración: que la app, la web y el servidor no encajen bien	Media	Alto	Acordar antes qué datos se envían y probar todo junto al final de cada sprint	Dedicar más tiempo del sprint 9 a arreglarlo y apuntar los errores en Trello
Equipo: que uno de los dos no pueda hacer su parte (enfermedad, exámenes…)	Media	Alto	Tener todo en GitHub y explicarnos lo que hace cada uno	El otro sigue esa parte y quitamos alguna ampliación
Dependencia de terceros: no encontrar una peluquería que nos ayude	Media	Medio	Preguntar a varias peluquerías desde el sprint 1	Hacer las pruebas con familiares y compañeros
Hardware: que falle el móvil o el portátil en la demo	Baja	Alto	Tener la app instalada en dos móviles y el proyecto en los dos portátiles	Usar el emulador, el otro portátil o el vídeo grabado de la demo
Pérdida de datos o código	Baja	Alto	Subir el código a GitHub a menudo y hacer copias de la base de datos cada semana	Recuperar el código de GitHub y la última copia de la base de datos
Permisos: que no nos dejen usar un servidor del instituto	Media	Bajo	Preguntarlo pronto al profesorado	Usar un portátil como servidor

20. Plan de pruebas

- Definir pruebas funcionales.
- Definir pruebas de integración.
- Definir pruebas de usabilidad.
- Definir pruebas de seguridad.
- Definir pruebas de rendimiento si tienen sentido.
- Definir pruebas ante fallos o recuperación.
- Explicar qué resultado se espera en cada tipo de prueba.

Iremos probando cada parte a medida que la hagamos, y en los sprints 11 y 12 haremos las pruebas completas. Apuntaremos cada prueba en una tabla con lo que esperábamos, lo que ha pasado y si está bien o no.

Tipo de prueba	Qué probaremos	Resultado esperado
Funcionales	Registrarse, iniciar sesión, crear un perro, reservar una cita, ver la agenda, añadir y borrar clientes, cambiar precios y horario	Cada cosa hace lo que tiene que hacer y los datos se guardan bien

Integración	Reservar desde la app y mirar la web; cambiar un precio o cerrar un día en la web y mirar la app	Lo que se hace en una parte aparece en la otra
Usabilidad	Pedir a 3 o 4 personas (familia, compañeros y la peluquería) que reserven una cita sin ayuda	Lo consiguen sin preguntarnos y en pocos minutos; apuntamos lo que les cueste
Seguridad	Contraseña incorrecta, cliente intentando ver datos de otro o entrar en la web del dueño, revisar que las contraseñas están cifradas	No deja entrar ni ver datos que no son suyos
Rendimiento	Usar la app con muchas citas guardadas (por ejemplo, 200 citas de prueba)	Las pantallas siguen cargando en 2 o 3 segundos
Fallos y recuperación	Apagar el servidor con la app abierta; dos personas reservando la misma hora a la vez; borrar la base de datos de prueba y recuperar la copia	La app muestra un mensaje de error y no se cierra; solo una reserva se guarda; la copia se recupera entera

22. Criterios de aceptación

- Definir entre 5 y 10 condiciones objetivas que deberán cumplirse para considerar terminado el proyecto.
- Los criterios deben poder demostrarse o verificarse.
- Deben estar relacionados con las funcionalidades y objetivos definidos al inicio.

Consideraremos el proyecto terminado cuando se cumplan estas condiciones, y todas se podrán enseñar en directo:

1. Un cliente nuevo se puede registrar e iniciar sesión desde la app en un móvil Android
2. El cliente puede crear la ficha de su perro con nombre, raza y tamaño, y verla después
3. Al reservar, la app solo enseña las horas libres según el horario de la peluquería y las citas que ya hay
4. El cliente puede reservar una cita en cuatro pasos o menos, sin llamar a nadie
5. La cita reservada desde la app aparece en la agenda de la web del dueño
6. No se pueden reservar dos citas a la misma hora
7. El dueño puede añadir, editar y borrar clientes desde la web
8. Si el dueño cambia un precio o cierra un día en la web, se ve el cambio en la app
9. Un cliente no puede ver los datos de otro cliente ni entrar en la web del dueño, y las contraseñas están cifradas en la base de datos
10. El código está en GitHub y hay un manual para instalarlo y otro para usarlo

23. Entregables finales

- Producto o sistema funcional.

- Código fuente y configuraciones necesarias.
- Base de datos y scripts si corresponde.
- Manual de instalación o despliegue.
- Manual de usuario.
- Documentación técnica.
- Diagrama de arquitectura final.
- Plan de pruebas y resultados.
- Presentación final.
- Demostración en directo.
- Cualquier otro entregable específico del proyecto.

Entregable	Qué incluye
Producto funcional	La app de Android (el archivo .apk para instalarla), la web del dueño y el servidor funcionando
Código fuente y configuración	Todo el código en GitHub (carpetas app, web y servidor) y un archivo de configuración de ejemplo, sin contraseñas reales
Base de datos y scripts	El script SQL que crea las tablas y otro con datos de prueba inventados
Manual de instalación	Los pasos para instalar MySQL, poner en marcha el servidor, abrir la web e instalar la app en el móvil
Manual de usuario	Una guía con capturas para el cliente (cómo reservar) y para el dueño (cómo usar la agenda y la gestión)
Documentación técnica	Este documento actualizado, con la lista de peticiones del servidor (API) y cómo está organizado el código
Diagrama de arquitectura final	El diagrama del apartado 11 actualizado con cómo ha quedado al final
Diagrama entidad-relación	El modelo de la base de datos final
Plan de pruebas y resultados	La tabla de pruebas del apartado 20 con lo que ha salido en cada una
Presentación final	Las diapositivas para explicar el proyecto
Demostración en directo	El recorrido del apartado 24, y un vídeo grabado como plan B
Prototipo en Figma	Los bocetos de las pantallas de la app y de la web

24. Demostración final

- Describir qué se enseñará en directo.
- Definir el recorrido de la demostración.
- Indicar qué entorno, datos, dispositivos o servicios serán necesarios.
- Definir qué parte demostrará mejor el valor del proyecto.
- Preparar un plan alternativo si falla internet, un servidor o un dispositivo.

En la demo enseñaremos lo más importante de CitaCan: un cliente reserva desde el móvil y la cita aparece en la web del dueño.

Recorrido de la demo:

1. Albert, desde el móvil, se registra como un cliente nuevo y crea la ficha de su perro.

2. Elige el servicio "Baño + corte" y la app le dice cuánto durará.

3. Elige un día y le salen solo las horas libres. Reserva una.

4. Pol, en el ordenador, entra en la web del dueño y la cita aparece en la agenda.

5. Pol cambia el precio de un servicio y cierra un día. Albert actualiza la app y se ve el cambio.

6. Intentamos reservar la misma hora otra vez y no deja.

7. Si hemos hecho ampliaciones (cancelar una cita, ingresos del mes…), las enseñamos al final.

Lo que necesitaremos: un portátil con el servidor y la base de datos en marcha, un móvil Android con la app instalada, conexión a internet o una red wifi para los dos, el proyector y una base de datos con datos de prueba inventados (unos cuantos clientes, perros y citas ya creados).

Lo que mejor enseña el valor del proyecto: el momento en que se reserva en el móvil y la cita aparece sola en la web, porque es justo el problema que queremos resolver: que el dueño no tenga que coger el teléfono.

25. Control de cambios

- Documentar cualquier cambio importante respecto al documento inicial.
- Indicar qué se cambia y por qué.
- Explicar el impacto del cambio sobre el alcance o los plazos.
- Mantener un histórico de decisiones importantes durante el curso.

Cada vez que cambiemos algo importante respecto a este documento, lo apuntaremos en esta tabla con la fecha, qué se cambia, por qué y cómo afecta al proyecto. Así tendremos un historial de todas las decisiones del curso. Lo revisaremos en la reunión del final de cada sprint.

Fecha	Qué se cambia	Por qué	Impacto en el alcance o los plazos
04/10/2026	Elegimos Java con Spring Boot para el servidor, en vez de Node.js	Es el mismo lenguaje que la app y el que damos en clase	Ninguno, solo aprendemos un lenguaje
04/10/2026	La web del dueño se hará sin React, solo con HTML, CSS y JavaScript	Para no tener que aprender otra herramienta	Ninguno, ahorra tiempo
04/10/2026	"Buscar peluquerías cercanas" pasa a ser una ampliación	El MVP será para una sola peluquería	Reduce el alcance del MVP

26. Autoevaluación de viabilidad

- Valorar si el MVP puede realizarse con los conocimientos disponibles.
- Comprobar si las tecnologías principales son asumibles para el equipo.
- Valorar si el proyecto depende de recursos difíciles de conseguir.
- Comprobar si puede demostrarse de forma real al final del curso.

- Valorar si el alcance cabe razonablemente en el tiempo disponible.
- Concluir si el proyecto es viable tal como está, si debe reducir alcance o si necesita replantearse. Ç

Creemos que el proyecto es viable si nos centramos en el MVP, aunque sabemos que nos costará porque es la primera aplicación que hacemos

- ¿Se puede hacer el MVP con lo que sabemos? Todavía no del todo. Nunca hemos hecho una app ni un servidor, así que tendremos que aprender bastante por nuestra cuenta con tutoriales, la documentación y la ayuda del profesorado. Por eso hemos dejado un sprint entero (el 3) para preparar las herramientas y hacer pruebas pequeñas antes de empezar en serio

- ¿Las tecnologías son asumibles? Sí, porque hemos elegido las más parecidas a lo que damos en clase: Java para la app (con Android Studio) y para el servidor (con Spring Boot), MySQL para la base de datos, y HTML, CSS y JavaScript para la web del dueño. Así aprovechamos lo que vemos en Programación, Bases de Datos y Lenguajes de Marcas. React lo dejamos fuera para no tener que aprender otra cosa más

- ¿Dependemos de recursos difíciles de conseguir? No. Todo lo que usamos es gratis (Android Studio, MySQL, GitHub, Figma) y ya tenemos portátiles y un móvil Android. Lo único que no depende de nosotros es que una peluquería quiera hacer la prueba, y si no lo conseguimos, haremos las pruebas con familiares y compañeros

- ¿Se puede enseñar de verdad al final del curso? Sí. Con un portátil y un móvil podemos enseñar el recorrido completo en directo

- ¿Cabe en el tiempo que tenemos? El MVP sí, porque está previsto acabarlo en el sprint 9 y quedan cuatro sprints de margen. Las ampliaciones (recordatorios, ingresos, fotos) puede que no nos dé tiempo a hacerlas todas

Conclusión: el proyecto es viable tal como está planteado, siempre que hagamos primero el MVP y dejemos las ampliaciones para el final. Si a mitad de curso vemos que vamos con retraso, lo primero que quitaremos serán las ampliaciones, no el MVP

27. Anexos obligatorios o recomendados

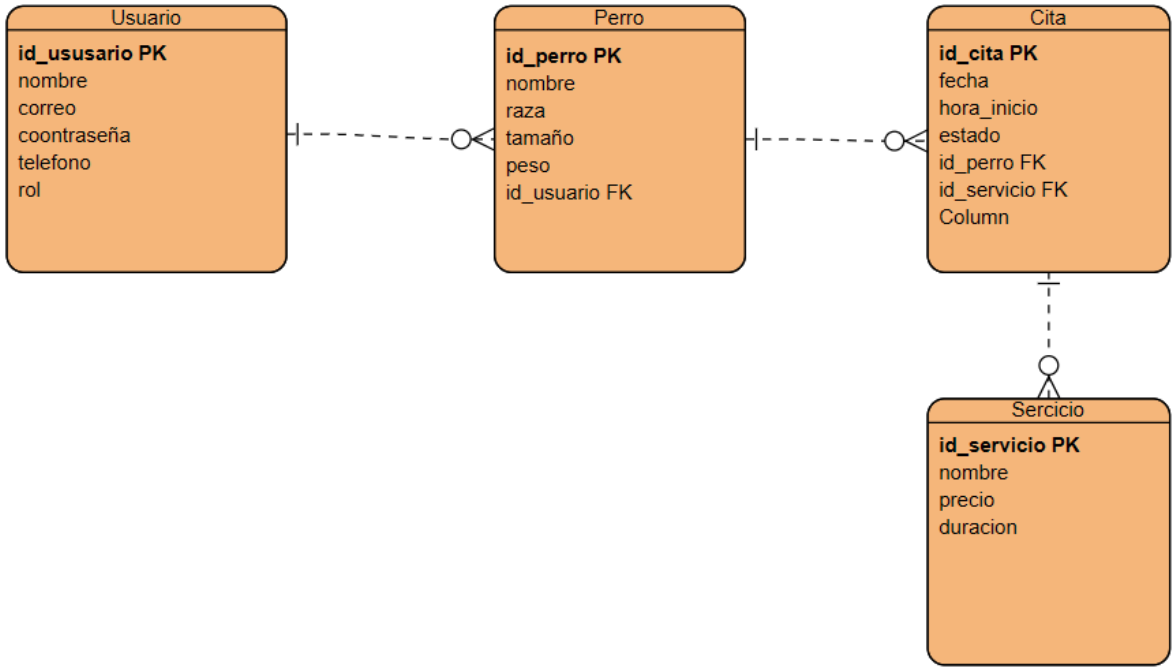
- Bocetos, wireframes o prototipo.
- Diagrama de arquitectura.
- Modelo de datos o diagrama entidad-relación.
- Esquema de red o infraestructura cuando corresponda.
- Pruebas de concepto, capturas o investigación previa.
- Enlaces al repositorio y a las herramientas de gestión del proyecto.

Estos son los anexos que acompañarán al documento. Algunos ya están hechos y otros los iremos añadiendo durante el curso.

Anexo	Estado	Dónde está
Bocetos y prototipo de las pantallas	Primer boceto hecho; el prototipo en Figma se hace en el sprint 2	Presentación inicial y Figma
Diagrama de arquitectura	Hecho	Apartado 11
Modelo de datos (diagrama entidad-relación)	Primera versión hecha	Abajo, en este apartado

Esquema de red	No hace falta uno aparte: es una red wifi sencilla con el móvil y el servidor	Apartado 13
Investigación previa	Pendiente: entrevista a una peluquería canina en el sprint 1	Se añadirá aquí
Enlace al repositorio de GitHub	Pendiente: se crea en el sprint 3	Enlace: [añadir]
Enlace al tablero de Trello	Pendiente: se crea en el sprint 1	Enlace: [añadir]

- Modelo de datos (primera versión):



CRITERIO DE ENTREGA

El documento final debe estar redactado de forma profesional, con títulos claros, lenguaje técnico cuando corresponda, imágenes o diagramas útiles y sin respuestas de una sola frase. Debe permitir entender el proyecto completo sin necesidad de escuchar una explicación oral del equipo.

El objetivo no es hacer muchas páginas por hacerlas, sino dejar suficientemente definido qué vais a construir y disponer de una referencia real para comprobar durante el curso si el proyecto avanza o se está desviando de lo planificado.